

Fly GACA — Process Workflow Documentation

SECTION 06-operations-it

TYPE document

STATUS **active**

OWNER Founder

UPDATED 2026-08-19

Location: `06-operations-it/diagrams/` **Generated:** 2026-06-14 · **Re-based on the Cloud Run / Postgres stack:** 2026-08-19 **Covers:** the three core operational processes illustrated in the SVG diagrams in this folder. **Context docs:** `fly-gaca-review-action-plan.md` (especially §2.1, §4.1, §4.2), `captadel-plan.md`, `06-operations-it/spec-freshness-pipeline.md`, `06-operations-it/diff-tracker-scope.md`, `06-operations-it/runbooks/runbook-source-updates.md`, `06-operations-it/hosting-facts.md`.

Stack note. Workflows 2 and 3 name code paths. Those paths moved when the product left Firebase: there are no Cloud Functions, no Firestore and no App Check. The API is a single Express service on **Cloud Run** (`me-central2`, Dammam) over **Cloud SQL Postgres**, the SPA and corpus are served from **Cloud Storage** buckets, and the web pages are React routes, not standalone `.html` files. The tables below use the real paths — see `hosting-facts.md`.

1. Licensing Journey (`licensing-journey.svg`)

Purpose

A reference map of the Saudi civil pilot licensing pathway from entry prerequisites through PPL issue, plus the foreign licence conversion route. This is the journey Fly GACA's Guides cover. The diagram is an educational aid — GACA's official publications are always the authoritative source.

Phases and steps

Phase	Step	Key regulatory reference
1 — Entry	Age 17+, Saudi nationality or valid residency, basic education	GACAR Part 61 Subpart B
2 — Medical	Class 2 Medical Certificate from a GACA-approved AME (vision, cardiovascular, hearing/ENT/mental)	GACAR Part 67; valid 24 months for applicants under 40
3 — Student Pilot	Application to GACA, dual instruction begins with CFI, solo endorsement required	GACAR 61.87, Part 61 Subpart C
4 — Training	Ground school, written knowledge test (70% pass mark), ICAO Language Proficiency Level 4 English test, 45 flight hours (incl. 10 hrs solo PIC)	GACAR Part 61 Subpart E; ICAO Annex 1
5 — Checkride	Oral exam + flight test with a GACA-designated examiner; fail means further instruction and re-test	GACAR Part 61 Subpart E, ACS standards

Phase	Step	Key regulatory reference
6a — PPL Issued	GACAR Part 61 PPL, VFR day/night privileges; medical renewal due every 24 months; next step: hour-building toward CPL/IR	GACAR Part 61
6b — Conversion	Holders of FAA/EASA/ICAO licences may convert under GACAR 61.77; requires English proficiency + knowledge test + differences training if needed	GACAR 61.77

Owner / role

Content team (keeps guide pages aligned with phases); editorial owner (ensures GACAR Part 61 reference is current each AIRAC cycle — see Workflow 2).

Failure modes

- Guide page refers to a superseded requirement (e.g., old hour minimums) — caught by AIRAC editorial sync (Workflow 2).
- Conversion path details change without a guide update — must be flagged in the per-cycle checklist.

2. AIRAC Editorial Sync ()

Purpose

The per-AIRAC-cycle process that keeps every regulation, aerodrome, chart, and handbook page on Fly GACA current and visibly stamped. This is the §2.1 trust-gap mitigation from the action plan. The action plan is explicit: a stamp that says "sync'd 8 months ago" is more damaging than no stamp. Currency is a product function, not a UI feature, and it requires a named editorial owner.

Cadence

The AIP changes every **28-day AIRAC cycle**, effective each Thursday. GACAR Parts amend irregularly. The pipeline is designed to run weekly (Monday) and every Thursday to align with AIRAC effective dates.

Pipeline stages

Stage	What happens	Key file / code
A. Detect	Fetch canonical GACA pages; discover all document links; fingerprint the set	<code>public/data/sources.json</code>
B. Hash + diff	Hash each Part/doc; compare against the persisted fingerprint; flag changed items	<code>npm run sync:gaca</code> → <code>scripts/sync-gaca.mjs</code> , <code>scripts/lib/sync-merge.mjs</code>

Stage	What happens	Key file / code
C. Re-ingest	Merge metadata deltas; re-normalise; re-chunk and re-embed incrementally (not a full rebuild)	<code>npm run sync:gaca:apply</code> , <code>data:normalize</code> , <code>parse:regulations</code> , <code>build:chunks</code> → <code>public/data/rag-chunks.json</code> , <code>embeddings:upsert</code> (Supabase pgvector)
D. Stamp	Write/refresh provenance block (<code>fetch_at</code> , <code>verified_cycle</code> , <code>amendment</code> , AIRAC effective date)	<code>public/data/source-status.json</code>
E. Eval gate	Run eval harness; any citation that no longer resolves to a live section fails the build and blocks publish	<code>ay2m/Captain-Adel</code> <code>evals/</code>
F. Publish	Sync <code>public/data/</code> to the corpus bucket (served network-first, so clients refresh without an app deploy); roll a Cloud Run revision so the API picks up the rebuilt <code>rag-chunks.json</code> ; tag the revision with the AIRAC cycle	<code>gcloud storage rsync</code> , <code>gcloud run deploy</code>

If **nothing changed** at stage B, the pipeline logs "current" and exits — no re-ingest, no re-publish.

[!WARNING] The scheduler half of this workflow is not currently running. `sources.json` and `source-status.json` still name `scripts/update-sources.js` and an `update-sources.yml` workflow as their owner; neither exists in `ay2m/FlyGACA` today, and `source-status.json` was last generated 2026-06-15. Stages A–F are all runnable by hand; nothing triggers them. Until a cron (VPS or Cloud Scheduler) is re-created, the freshness stamp in §"User-visible outputs" below is a promise the pipeline is not keeping — which is precisely the worst failure mode named further down this section.

Editorial owner actions (human, per checklist)

The automated pipeline detects and re-ingests changes. The editorial owner's role is to **confirm material significance and sign off**:

1. Review the auto-detected changes log.
2. Confirm changed Parts against the official GACA publication.
3. Review the diff for substantive regulatory amendments (not just formatting).
4. Update reader-page prose if needed (e.g., a changed hour minimum in a guide).
5. Sign off the freshness stamp (approve the `verified_cycle` update).

SLAs (from `spec-freshness-pipeline.md`):

- AIP-class (aerodromes, charts, AIS): re-verified within **72 hours** of each new AIRAC cycle.
- GACAR Parts: re-ingested within **7 days** of a detected amendment.
- Reference shelf (handbooks, ICAO, FAA): best-effort, monthly diff.

User-visible outputs

Output	Trigger	Appearance
Freshness stamp	Every reader page	"GACAR Part 61 — Amdt 3, verified AIRAC 2026-05" + green check
Staleness banner	<code>verified_cycle</code> older than current AIRAC for AIP-class item	Amber flag: "verification pending for the current cycle"
Library status line	Always visible	"Corpus current as of AIRAC YYYY-CC"
Captain Adel citation datestamp	Every RAG answer	Amendment + AIRAC stamp on each cited source

Failure modes

Failure	Consequence	Mitigation
GACA site returns 0 docs (blocked / 403)	No content wiped (fail-safe); alert owner	Soft warning (exit 0) on network trouble; positive change required to act
Eval gate fails after re-ingest	Publish blocked; owner alerted	Manual review and fix before next deploy
Stamp says "current" when actually stale	Worst failure — breaks the trust contract	Automated <code>verified_cycle</code> check; staleness banner auto-triggers if cycle drifts
Editorial owner misses the 72-hour AIP window	Staleness banner shown to users	Checklist alert; escalation to backup owner

Action plan reference

§2.1 (trust and authority gap), §4.1 (currency stamps + AIRAC flags), `spec-freshness-pipeline.md`, `runbook-source-updates.md`.

3. Captain Adel RAG Fallback (`captain-adel-fallback.svg`)

Purpose

The decision flow for Captain Adel's RAG answer pipeline, with the strict low-confidence fallback mandated by §4.2 of the action plan. The core principle: when retrieval confidence is low, Captain Adel **must refuse and redirect to the exact GACAR text** rather than guess. A confident wrong answer with a plausible citation is worse than a refusal. Conservative refusal is a feature, not a failure.

Pipeline steps

Step	Description	Code location
1. User question	Regulatory or aeronautical query arrives from the <code>/chat</code> route → <code>POST /api/chat</code> on the Cloud Run service. The gateway resolves the session (cookie, or bearer token from the native shell), meters the caller, and preserves the SSE contract	<code>src/pages/chat/</code> , <code>server/src/gateway.ts</code>
2. Scope + safety pre-check	Is this a GACAR/aviation question? Injection / role-confusion detection. Out-of-scope → immediate refuse	<code>server/src/captain-adel-prompt.ts</code>
3. BM25 retrieval	Lexical search over the GACAR chunk index (29,165 chunks as of the 2026-06-13 build); <code>RETRIEVE_K</code> passages, with full legal lineage (document → subpart → section → paragraph) on each. Dense/hybrid retrieval is the seam behind the retriever interface, not yet the default	<code>server/src/corpus.ts</code> , <code>public/data/rag-chunks.json</code>
4. Confidence decision	Grounding is computed server-side from the BM25 score, never trusted to the model: below <code>REFUSE_SCORE</code> → refuse without calling Gemini at all; below <code>GROUNDED_SCORE</code> → answer but flag "partially grounded"	<code>server/src/captain-adel.ts</code>
5. Gemini generation	Gemini via Genkit — <code>gemini-2.5-flash</code> , <code>gemini-2.5-pro</code> for the Pro tier — under the system prompt; answer constrained to retrieved passage text; must cite Part + section + amendment; token deltas streamed as SSE	<code>server/src/captain-adel.ts</code> , <code>server/src/sse.ts</code>
6. Citation faithfulness guard	Score every answer's claims against cited section text; score ≥ 0.8 to pass; fail → strip unsupported claims or refuse with partial grounding	<code>ay2m/Captain-Adel/evals/checks/citation-faithfulness.js</code>
7. Answer delivered	Streamed response with cited section, retrieved source snippet (not just a link), and amendment + AIRAC stamp; clickable deep-link into the Library reader	<code>src/pages/chat/</code> , <code>src/calculator/chat/</code>

The strict fallback (§4.2 — low confidence path)

When retrieval confidence is below threshold:

1. Do **not** attempt an answer.
2. State plainly that the question cannot be verified against GACAR with sufficient confidence.
3. Link the user to the exact GACAR Part(s) most likely relevant.
4. Never paraphrase a regulatory limit that cannot be grounded in a retrieved passage.

Canonical refusal template:

"I can't verify this against GACAR with sufficient confidence. For authoritative guidance, refer to: [exact GACAR Part link]."

Refusal taxonomy (docs/refusal-taxonomy.md)

Six refusal categories, each with canonical language and scored in the eval suite:

1. Out of scope (non-GACAR topic)
2. Ambiguous question (cannot determine which GACAR Part applies)
3. Low retrieval confidence (main fallback path)
4. Conflicting source passages (two retrieved sections contradict)
5. Missing or broken citation (passage cannot be linked back to a live section)
6. Injection or role-confusion attack detected

Eval harness gates (from captadel-plan.md Wave 1)

The system is only as trustworthy as its eval coverage. Eval cases required before each version ships:

Case class	Target count	Purpose
EN knowledge	50+	Baseline correctness
AR knowledge	30+	Arabic parity (within 5% of EN pass rate)
Adversarial (injection, role-confusion)	20+	Refusal robustness
Refusal calibration	10+	Correct refusal on low-confidence questions
Staleness (superseded citation)	New class	Hard failure — never cite a superseded source

A CI eval gate is required on all PRs touching the Captain Adel flow (server/src/captain-adel*.ts , server/src/corpus.ts) and on ay2m/Captain-Adel . A refusal-calibration regression is an unconditional block.

Supporting operational features

- **Quota, in Postgres:** the daily counter is a chat_usage (user_key, day, count) row incremented atomically — free tier 5 questions/day (CHAT_FREE_DAILY_LIMIT), anonymous callers metered by hashed IP (ANON_DAILY_LIMIT), Pro unlimited, purchased credits spent from chat_credits . Policy lives in the pure chat-quota-core.ts and is mirrored client-side by src/calc/chat/chatQuota.ts ; tests/client-server-mirrors.test.ts fails the build if the two drift.
- **Rate limiting:** express-rate-limit plus the pure fixed-window rate-limit-core.ts . State is per-instance memory, so with max-instances=10 the real ceiling is limit × instances — acceptable for a cost control, and deliberately **not** a security boundary. The hard ceiling is the Postgres quota above.

- **Abuse controls:** session auth (HS256 JWT cookie) or an API key for the licensed `/v1/ask` tier (`api-key-core.ts` / `api-tier-core.ts`), a request-body cap, and the CORS allowlist in `gateway-core.ts`. *(There is no App Check — that was a Firebase primitive and left with it.)*
- **Structured observability:** per-turn metrics — latency, sources retrieved, refusal rate, faithfulness score, model — via Cloud Run request logs and `analytics-core.ts`; feeds real-world eval seeding.
- **ALLaM (target):** in-Kingdom model for PDPL compliance and cost trajectory; AR eval pass rate must stay within 5% of EN.

Owner / role

Captain Adel product owner (system prompt, refusal taxonomy, eval case ownership); engineering (retrieval, pipeline, CI gate); editorial owner (ensures corpus freshness so retrieval has current chunks to draw from — feeds into Workflow 2).

Failure modes

Failure	Consequence	Mitigation
Low-confidence answer slips through without refusal	Wrong operational guidance delivered to user; reputational damage	CI gate: refusal-calibration eval cases block regression
Citation faithfulness guard miscalibrated (too strict)	Legitimate answers refused; user frustration	Human-labelled subset quarterly review of judge calibration
Corpus stale — retrieval returns superseded chunks	Answer correct per old regulation	Staleness eval case class + AIRAC sync (Workflow 2) — superseded chunks excluded from index
Injection attack rewrites system prompt	Adel behaves as non-Adel; trust destroyed	Pre-check step + adversarial eval cases; runtime injection logging
Cost spike from runaway eval or high traffic	Budget overrun	Per-caller quota (<code>chat_usage</code>) + API-tier monthly caps; <code>max-instances</code> on the Cloud Run service; eval concurrency limits; a Cloud Billing budget alert on the GCP project

Action plan reference

§2.3 (AI hallucination risk), §4.2 (strict fallback protocol), `captadel-plan.md` §1b (citation-faithfulness) and Wave 1 (eval foundation), `runbook-captain-adel.md`.

Cross-workflow dependencies

```
Workflow 2 (AIRAC sync)
└> keeps corpus fresh
    └> Workflow 3 (Captain Adel) retrieves from fresh index
        └> Workflow 1 (Licensing Journey) guides reference current GACAR Parts
```

The trust proposition of Fly GACA depends on all three workflows running correctly. Workflow 2 is the foundation: if the corpus is stale, both Captain Adel (Workflow 3) and the Guides (Workflow 1) become unreliable regardless of how well their own logic is implemented.